

Universidad Nacional de Río Cuarto
Facultad de Ciencias Exactas, Físico-Químicas y Naturales
Departamento de Computación
Licenciatura en Ciencias de la Computación

Bases de Datos II

Trabajo Práctico N.º 2

DCL: usuarios, roles y privilegios

Alumno: Tomás Rodeghiero
Año: 2026

Índice

1. Introducción	2
2. Ejercicio 1: Usuarios y privilegios en MySQL	2
3. Ejercicio 2: Conexión por host y límites de recursos	4
4. Ejercicio 3: Roles y herencia en PostgreSQL	6
5. Ejercicio 4: Perfiles, vistas y delegación en Oracle	9
6. Conclusión	13

1. Introducción

Este práctico aborda DCL, el subconjunto de SQL dedicado al control de acceso. Los cuatro ejercicios ejercitan las tres operaciones centrales del subsistema de seguridad: crear usuarios y roles, otorgar privilegios y revocarlos. Cada ejercicio se resuelve sobre una de las bases definidas en el Práctico 1 y en el motor que pide la cátedra (MySQL, PostgreSQL u Oracle), lo que también permite contrastar las diferencias de implementación entre motores.

Cada ejercicio sigue siempre la misma estructura: **Consigna** (texto literal del PDF), **Idea** (lectura natural y concepto DCL detrás), **Solución** (SQL en el motor que pide el enunciado), **Por qué funciona** (lógica paso a paso) y **En MySQL 8+** (equivalente portable a MySQL cuando el ejercicio pide otro motor). Los ejercicios 1 y 2 ya están en MySQL: en ellos el bloque “En MySQL 8+” anota qué notas de portabilidad inversa son útiles para entender por qué la misma resolución no se vería igual en PostgreSQL o en Oracle.

A lo largo del documento se priorizan dos criterios. Primero, otorgar siempre el privilegio mínimo necesario: cuando un inciso pide acceder solo a ciertas columnas, se usa la sintaxis a nivel de columna en lugar de un permiso amplio sobre la tabla. Segundo, justificar el uso (o la ausencia) de `WITH GRANT OPTION`: solo aparece cuando el enunciado pide explícitamente que el usuario pueda delegar el privilegio.

2. Ejercicio 1: Usuarios y privilegios en MySQL

Consigna

Dado el ejercicio 1)a) de la práctica 1) (base de datos clientes, productos, etc), cree los usuarios: vendedor1, vendedor2 y administrador, tener en cuenta:

- El vendedor1 posee el privilegio de insertar datos en la tabla cliente. Este usuario no tiene permitido otorgar estos privilegios a otros usuarios.
- El vendedor2 sólo tiene privilegios para insertar en la tabla factura y para seleccionar los campos de la tabla producto.
- El administrador tiene los privilegios de actualización del campo descripción de la tabla producto, pudiendo otorgar este privilegio a otros usuarios.
- Tener en cuenta que todos los usuarios cuentan con el privilegio de borrar los datos de la tabla cliente.
- Verificar que los permisos asignados a los usuarios en los incisos anteriores estén bien definidos.
- Al finalizar revoque todos los privilegios del vendedor2.

Nota: Resolver utilizando el motor de MYSQL.

Idea

DCL puro en MySQL: crear tres cuentas, otorgar privilegios distintos (con o sin delegación), verificar y revocar. Los conceptos centrales son `CREATE USER`, `GRANT` a nivel tabla y a nivel columna, `WITH GRANT OPTION` (que en MySQL se separa visualmente al final del `GRANT`) y `REVOKE`. La base de trabajo es `practico1a_mysql`, la del Ejercicio 1.a) del Práctico 1.

Solución

Conexión al servidor. Las sentencias DCL requieren un usuario con privilegios administrativos, típicamente `root`.

```
mysql -h localhost -P 3306 -u root -p
```

```
SHOW DATABASES;
USE practico1a_mysql;
SELECT DATABASE(), CURRENT_USER();
```

Creación de usuarios. Se crean con alcance '@localhost'; las contraseñas cumplen la política por defecto de validate_password en MySQL 8.

```
CREATE USER IF NOT EXISTS 'vendedor1'@'localhost' IDENTIFIED BY 'Vendedor1_2026!';
CREATE USER IF NOT EXISTS 'vendedor2'@'localhost' IDENTIFIED BY 'Vendedor2_2026!';
CREATE USER IF NOT EXISTS 'administrador'@'localhost' IDENTIFIED BY 'Admin_2026!';
```

Otorgamiento de privilegios. Cada GRANT traduce literalmente un inciso del enunciado.

```
-- a) vendedor1: INSERT en cliente, sin posibilidad de delegar.
-- Se omite WITH GRANT OPTION porque el enunciado lo prohíbe.
GRANT INSERT ON practico1a_mysql.cliente TO 'vendedor1'@'localhost';

-- b) vendedor2: INSERT en factura y SELECT a nivel de columna en producto.
GRANT INSERT ON practico1a_mysql.factura TO 'vendedor2'@'localhost';
GRANT SELECT (cod_producto, descripcion, precio, stock_minimo, stock_maximo, cantidad)
ON practico1a_mysql.producto
TO 'vendedor2'@'localhost';

-- c) administrador: UPDATE solo sobre la columna descripcion en producto.
-- WITH GRANT OPTION porque el enunciado pide que pueda delegar.
GRANT UPDATE (descripcion)
ON practico1a_mysql.producto
TO 'administrador'@'localhost'
WITH GRANT OPTION;

-- d) DELETE sobre cliente para los tres usuarios.
GRANT DELETE ON practico1a_mysql.cliente
TO 'vendedor1'@'localhost', 'vendedor2'@'localhost', 'administrador'@'localhost';
```

Verificación (inciso e).

```
SHOW GRANTS FOR 'vendedor1'@'localhost';
SHOW GRANTS FOR 'vendedor2'@'localhost';
SHOW GRANTS FOR 'administrador'@'localhost';
```

Más allá de SHOW GRANTS, conviene loguearse con cada cuenta y comprobar:

- vendedor1: INSERT/DELETE sobre cliente funcionan; cualquier GRANT ... disparado por vendedor1 debe fallar (no tiene GRANT OPTION).
- vendedor2: INSERT en factura, SELECT sobre los campos permitidos de producto y DELETE en cliente funcionan.
- administrador: UPDATE de descripcion en producto funciona y, gracias a WITH GRANT OPTION, también puede correr un GRANT UPDATE (descripcion) ON ... TO otroUsuario.

Revocación de vendedor2 (inciso f).

```
REVOKE ALL PRIVILEGES, GRANT OPTION FROM 'vendedor2'@'localhost';
SHOW GRANTS FOR 'vendedor2'@'localhost';
```

Por qué funciona

- En MySQL, la identidad efectiva es el par `'usuario'@'host': 'vendedor1'@'localhost'` y `'vendedor1'@' %'` son cuentas distintas. Acotar al `localhost` es el caso pedido y reduce la superficie de ataque.
- `GRANT SELECT (col1, col2, ...)` y `GRANT UPDATE (col)` aplican el privilegio a las columnas listadas y no al resto de la tabla. Es la forma idiomática de modelar “puede ver/actualizar estos campos pero no otros” sin necesidad de crear vistas.
- `WITH GRANT OPTION` aparece solo en `administrador` porque es el único inciso que pide delegación. En `vendedor1` el enunciado lo prohíbe explícitamente, así que se omite.
- `REVOKE ALL PRIVILEGES, GRANT OPTION FROM ...` es la forma directa de dejar la cuenta sin permisos. Tras correrlo, `SHOW GRANTS` debe devolver únicamente `USAGE`, que indica que la cuenta sigue existiendo y puede conectarse, pero no opera sobre ninguna tabla.

En MySQL 8+

La consigna ya pide MySQL, así que la sección *Solución* es la versión MySQL 8+. Las únicas portabilidades inversas a recordar son: PostgreSQL no usa el par `'user'@'host'` (la separación de origen se modela con `pg_hba.conf`); y Oracle no admite `GRANT SELECT(col)` directo sobre tablas (en ese motor el inciso b) requeriría crear una vista, como se hace en el Ejercicio 4).

3. Ejercicio 2: Conexión por host y límites de recursos

Consigna

Teniendo en cuenta la base de datos del inciso b) del practico 1) (base de datos de automóviles, estacionamiento, etc), cree los usuarios “encargado_estacionamiento” el cual se conecta sólo desde la máquina servidor (localhost) y “cobrador” que se puede conectar desde cualquier máquina. Además, tenga en cuenta lo siguiente:

- a) El “cobrador” sólo puede consultar la tabla estacionamiento y realizar 3 conexiones/hora y 10 consultas/hora.
- b) El usuario “encargado_estacionamiento” tiene los privilegios de consultar todas las tablas de la base de datos y de insertar en la tabla parquímetro en dicha base de datos.
- c) Compruebe la correcta definición de permisos (de accesos al servidor y a las base de datos).
- d) Borre el privilegio de consulta del usuario “cobrador”.

Nota: Resolver utilizando el motor MYSQL.

Idea

Es DCL en MySQL con dos elementos extra respecto del Ejercicio 1: la restricción del host de origen (que en MySQL se modela directamente en el nombre de la cuenta) y la imposición de *cuotas por hora* sobre el `cobrador`, que se configuran con `ALTER USER ... WITH MAX_CONNECTIONS_PER_HOUR / MAX_QUERIES_PER_HOUR`. La base es `practico1b_mysql`.

Solución

Creación de usuarios con alcance de host.

```
-- Limpieza opcional para re-ejecutar de forma idempotente
DROP USER IF EXISTS 'encargado_estacionamiento'@'localhost';
DROP USER IF EXISTS 'cobrador'@' %';
```

```
-- encargado_estacionamiento: solo se conecta desde localhost
CREATE USER 'encargado_estacionamiento'@'localhost'
IDENTIFIED BY 'Encargado_2026!';

-- cobrador: puede conectarse desde cualquier maquina
CREATE USER 'cobrador'@'%'
IDENTIFIED BY 'Cobrador_2026!';
```

Otorgamiento de privilegios y límites.

```
-- a) cobrador: SELECT sobre estacionamiento y limites de uso por hora.
GRANT SELECT ON practico1b_mysql.estacionamiento
TO 'cobrador'@'%' ;

ALTER USER 'cobrador'@'%'
WITH
    MAX_CONNECTIONS_PER_HOUR 3
    MAX_QUERIES_PER_HOUR 10;

-- b) encargado_estacionamiento: SELECT sobre todas las tablas e
--     INSERT acotado a parquemetro.
GRANT SELECT ON practico1b_mysql.*
TO 'encargado_estacionamiento'@'localhost';

GRANT INSERT ON practico1b_mysql.parquemetro
TO 'encargado_estacionamiento'@'localhost';
```

Verificación (inciso c). Hay que mirar dos planos: privilegios efectivos sobre objetos y configuración de la cuenta (host y límites).

```
-- Permisos efectivos
SHOW GRANTS FOR 'cobrador'@'%' ;
SHOW GRANTS FOR 'encargado_estacionamiento'@'localhost';

-- Definicion de la cuenta: host de origen y limites declarados
SHOW CREATE USER 'cobrador'@'%' ;
SHOW CREATE USER 'encargado_estacionamiento'@'localhost';
```

Revocación del privilegio de consulta del cobrador (inciso d).

```
REVOKE SELECT ON practico1b_mysql.estacionamiento
FROM 'cobrador'@'%' ;

SHOW GRANTS FOR 'cobrador'@'%' ;
```

Por qué funciona

- El host del lado derecho del @ controla desde qué máquinas se acepta la conexión. 'localhost' solo permite acceso desde el servidor; '%' es el comodín que acepta cualquier origen.
- Las cuotas MAX_CONNECTIONS_PER_HOUR y MAX_QUERIES_PER_HOUR se aplican a la cuenta entera, no a un privilegio en particular. Por eso van en un ALTER USER ... WITH separado, después del GRANT.
- GRANT SELECT ON practico1b_mysql.* otorga lectura sobre todas las tablas del esquema; agregar GRANT INSERT ON practico1b_mysql.parquemetro en una sentencia adicional acota la escritura a una sola tabla.
- Después del REVOKE del inciso d), la cuenta del cobrador sigue existiendo y sus límites de recursos siguen vigentes, pero ya no tiene SELECT. SHOW GRANTS debe devolver únicamente

USAGE.

En MySQL 8+

La consigna ya pide MySQL: la *Solución* es la versión MySQL 8+. Cabe anotar que el modelo de *cuotas por hora* de este ejercicio (`MAX_CONNECTIONS_PER_HOUR`, `MAX_QUERIES_PER_HOUR`) es característico de MySQL; PostgreSQL no expone esa noción directa (se aproxima con `connection_limit` a nivel de rol o con extensiones externas), y Oracle se modela vía `PROFILE` con `SESSIONS_PER_USER` y `CONNECT_TIME` (como se ve en el Ejercicio 4).

4. Ejercicio 3: Roles y herencia en PostgreSQL

Consigna

Utilizando el motor de bases de datos PostgreSQL y teniendo en cuenta la base de datos del inciso c) del practico 1) (base de datos de automóviles, accidentes, etc) cree los usuarios “asesor”, “administrativo” y “encargado”, teniendo en cuenta:

- Todos los usuarios pertenecen al rol “empleado”. El rol de empleado puede consultar los campos apellido y nombre de la tabla cliente.
- El “asesor”, además de lo que el rol empleado le permite, puede consultar la tabla taller e insertar en la tabla accidente.
- El “administrativo” puede consultar la tabla automóvil.
- El usuario “encargado” tiene los privilegios de borrar todas las tablas y actualizar el campo tasa de la tabla categoría.

Idea

El concepto clave es el modelo de roles de PostgreSQL: en este motor “usuario” y “rol” son la misma entidad y se diferencian solo por si pueden iniciar sesión (`LOGIN`) o no (`NOLOGIN`). Eso permite resolver con naturalidad el pedido: `empleado` se modela como rol `NOLOGIN` (contenedor de privilegios), y `asesor`, `administrativo` y `encargado` son roles `LOGIN` que lo heredan. El esquema es `practico1c`.

Solución

Pre-chequeo del esquema.

```
SELECT schema_name
FROM information_schema.schemata
WHERE schema_name = 'practico1c';

SELECT tablename
FROM pg_tables
WHERE schemaname = 'practico1c'
ORDER BY tablename;
```

Creación del rol común y de los usuarios.

```
-- Limpieza opcional
DROP ROLE IF EXISTS asesor;
DROP ROLE IF EXISTS administrativo;
DROP ROLE IF EXISTS encargado;
DROP ROLE IF EXISTS empleado;

-- Rol comun sin posibilidad de iniciar sesion
```

```

CREATE ROLE empleado NOLOGIN;

-- Usuarios reales: roles con LOGIN y contraseña
CREATE ROLE asesor LOGIN PASSWORD 'Asesor_2026!';
CREATE ROLE administrativo LOGIN PASSWORD 'Administrativo_2026!';
CREATE ROLE encargado LOGIN PASSWORD 'Encargado_2026!';

-- Inciso a): todos los usuarios pertenecen al rol empleado
GRANT empleado TO asesor, administrativo, encargado;

```

Otorgamiento de privilegios.

```

-- a) empleado: SELECT a nivel de columnas (apellido, nombre) en cliente.
GRANT SELECT (apellido, nombre)
ON practico1c.cliente
TO empleado;

-- b) asesor: además de empleado, SELECT en taller e INSERT en accidente.
GRANT SELECT ON practico1c.taller TO asesor;
GRANT INSERT ON practico1c.accidente TO asesor;

-- c) administrativo: SELECT sobre la tabla automovil.
GRANT SELECT ON practico1c.automovil TO administrativo;

-- d) encargado: DELETE sobre todas las tablas y UPDATE solo de la columna tasa
-- en categoria. ALL TABLES IN SCHEMA evita listar tabla por tabla.
GRANT DELETE ON ALL TABLES IN SCHEMA practico1c TO encargado;
GRANT UPDATE (tasa) ON practico1c.categoria TO encargado;

```

Comprobación de permisos.

```

-- roles y capacidad de login
SELECT rolname, rolcanlogin
FROM pg_roles
WHERE rolname IN ('empleado', 'asesor', 'administrativo', 'encargado')
ORDER BY rolname;

-- membresía al rol empleado
SELECT r.rolname AS role, m.rolname AS member
FROM pg_auth_members am
JOIN pg_roles r ON r.oid = am.roleid
JOIN pg_roles m ON m.oid = am.member
WHERE r.rolname = 'empleado'
ORDER BY m.rolname;

-- permisos por tabla
SELECT grantee, table_name, privilege_type
FROM information_schema.role_table_grants
WHERE table_schema = 'practico1c'
AND grantee IN ('empleado', 'asesor', 'administrativo', 'encargado')
ORDER BY grantee, table_name, privilege_type;

-- permisos por columna
SELECT grantee, table_name, column_name, privilege_type
FROM information_schema.column_privileges
WHERE table_schema = 'practico1c'
AND grantee IN ('empleado', 'encargado')
AND (
    (table_name = 'cliente' AND column_name IN ('apellido', 'nombre'))
    OR
    (table_name = 'categoria' AND column_name = 'tasa')
)

```



```
ORDER BY grantee, table_name, column_name;

-- chequeo puntual con funciones de privilegios
SELECT
  has_table_privilege('asesor', 'practico1c.taller', 'SELECT') AS
    asesor_select_taller,
  has_table_privilege('asesor', 'practico1c.accidente', 'INSERT') AS
    asesor_insert_accidente,
  has_table_privilege('administrativo', 'practico1c.automovil', 'SELECT') AS
    administrativo_select_automovil,
  has_table_privilege('encargado', 'practico1c.cliente', 'DELETE') AS
    encargado_delete_cliente,
  has_column_privilege('encargado', 'practico1c.categoria', 'tasa', 'UPDATE') AS
    encargado_update_tasa,
  has_column_privilege('asesor', 'practico1c.cliente', 'apellido', 'SELECT') AS
    asesor_select_apellido_via_empleado,
  has_column_privilege('administrativo', 'practico1c.cliente', 'nombre', 'SELECT') AS
    administrativo_select_nombre_via_empleado;
```

Por qué funciona

- `CREATE ROLE empleado NOLOGIN` declara un contenedor de privilegios que no puede usarse para conectarse; las tres cuentas con `LOGIN` lo heredan vía `GRANT empleado TO ...`, que es *membership* de rol (no es un `GRANT` sobre objetos).
- Cuando `asesor` ejecuta una consulta, PostgreSQL combina los privilegios directos de `asesor` con los del rol `empleado`. Así, sin escribirlo explícitamente, los tres usuarios obtienen el `SELECT` (`apellido`, `nombre`) sobre `cliente`.
- `GRANT SELECT (col1, col2)` otorga el privilegio acotado a esas columnas; cualquier `SELECT *` fallaría porque incluye campos no autorizados.
- `GRANT DELETE ON ALL TABLES IN SCHEMA practico1c` es un atajo de PostgreSQL para iterar sobre todas las tablas existentes *al momento de ejecutar la sentencia*; los objetos creados después quedan fuera. Si interesa cubrirlos también, hay que usar `ALTER DEFAULT PRIVILEGES`.
- La verificación combina varias vistas: `pg_roles` (existencia y `LOGIN`), `pg_auth_members` (herencia), `information_schema.role_table_grants` y `column_privileges` (qué permisos están declarados) y las funciones `has_table_privilege` / `has_column_privilege`, que verifican el privilegio *efectivo* considerando la herencia.

En MySQL 8+

MySQL 8+ también soporta roles (introducidos en 8.0), así que la consigna se puede traducir bastante directa. Las diferencias más relevantes: los roles en MySQL no tienen `LOGIN/NOLOGIN` (todo rol es “de mentira” como cuenta de conexión salvo que se le otorgue contraseña); por defecto los roles otorgados a una cuenta están *inactivos* y hay que activarlos con `SET DEFAULT ROLE` para que la herencia se aplique al iniciar sesión; y no existe `GRANT ... ON ALL TABLES IN SCHEMA`, hay que listar tabla por tabla o generar las sentencias con un script.

```
-- Asumimos la base practico1c_mysql definida en el Practico 1
USE practico1c_mysql;

-- Limpieza opcional
DROP USER IF EXISTS 'asesor'@'localhost';
DROP USER IF EXISTS 'administrativo'@'localhost';
DROP USER IF EXISTS 'encargado'@'localhost';
DROP ROLE IF EXISTS 'empleado';
```

```

-- Rol comun
CREATE ROLE 'empleado';

-- Usuarios con LOGIN (todo usuario en MySQL puede loguearse)
CREATE USER 'asesor'@'localhost' IDENTIFIED BY 'Asesor_2026!';
CREATE USER 'administrativo'@'localhost' IDENTIFIED BY 'Administrativo_2026!';
CREATE USER 'encargado'@'localhost' IDENTIFIED BY 'Encargado_2026!';

-- Inciso a): el rol empleado consulta apellido y nombre de cliente,
--             y los tres usuarios pertenecen al rol.
GRANT SELECT (apellido, nombre)
ON practico1c_mysql.cliente
TO 'empleado';

GRANT 'empleado' TO 'asesor'@'localhost',
                  'administrativo'@'localhost',
                  'encargado'@'localhost';

-- Activacion automatica del rol al iniciar sesion (clave en MySQL).
SET DEFAULT ROLE 'empleado'
TO 'asesor'@'localhost',
   'administrativo'@'localhost',
   'encargado'@'localhost';

-- b) asesor: SELECT en taller, INSERT en accidente
GRANT SELECT ON practico1c_mysql.taller TO 'asesor'@'localhost';
GRANT INSERT ON practico1c_mysql.accidente TO 'asesor'@'localhost';

-- c) administrativo: SELECT en automovil
GRANT SELECT ON practico1c_mysql.automovil TO 'administrativo'@'localhost';

-- d) encargado: DELETE sobre TODAS las tablas (sin "ALL TABLES IN SCHEMA"
--             en MySQL: hay que enumerarlas) y UPDATE de tasa en categoria.
GRANT DELETE ON practico1c_mysql.cliente TO 'encargado'@'localhost';
GRANT DELETE ON practico1c_mysql.categoria TO 'encargado'@'localhost';
GRANT DELETE ON practico1c_mysql.taller TO 'encargado'@'localhost';
GRANT DELETE ON practico1c_mysql.automovil TO 'encargado'@'localhost';
GRANT DELETE ON practico1c_mysql.accidente TO 'encargado'@'localhost';

GRANT UPDATE (tasa) ON practico1c_mysql.categoria TO 'encargado'@'localhost';

-- Verificacion
SHOW GRANTS FOR 'asesor'@'localhost' USING 'empleado';
SHOW GRANTS FOR 'administrativo'@'localhost' USING 'empleado';
SHOW GRANTS FOR 'encargado'@'localhost' USING 'empleado';

```

Qué cambia respecto de PostgreSQL: (i) en MySQL no hay NOLOGIN, el aislamiento del rol como “no usable para conectarse” se basa en no asignarle contraseña ni usarlo como cuenta; (ii) sin SET DEFAULT ROLE, los privilegios del rol no se aplican hasta que el usuario haga SET ROLE en su sesión; (iii) la cláusula USING ‘empleado’ en SHOW GRANTS sirve para ver los permisos efectivos considerando el rol activo.

5. Ejercicio 4: Perfiles, vistas y delegación en Oracle

Consigna

Resolver utilizando el motor Oracle (sqlplus).

Crear los usuarios “empleado1”, “empleado2” y “director”.

Los usuarios empleado 1 y 2 poseen el rol de “empleados”, el rol de empleados puede consultar información de la tabla automóvil y categoría de la base de datos del ejercicio 1) c) de la práctica 1) (base de datos de automóviles, accidentes, etc).

Tener en cuenta que empleado1 debe renovar su password.

Adicionalmente, el usuario director puede consultar todas las tablas y se le debe restringir el tiempo máximo de 20 minutos para estar conectado.

El usuario “empleado1” tiene el privilegio de borrar, actualizar y agregar los datos de la tabla accidente.

El usuario “empleado2” puede consultar, además por lo que su rol le permite, los campos apellido, nombre y dirección de la tabla cliente y puede pasar este privilegio a otros usuarios.

Idea

Tres aspectos específicos de Oracle entran en juego: PASSWORD EXPIRE para forzar renovación de contraseña, CREATE PROFILE ... CONNECT_TIME para limitar la duración total de la sesión, y la falta de GRANT SELECT(col) sobre tablas, que obliga a crear una vista para el inciso de empleado2. El esquema dueño de las tablas es PRACTIC01C.

Solución

Nota previa: Oracle y privilegios por columna. Oracle no admite GRANT SELECT (col1, col2) sobre una tabla; sí admite GRANT UPDATE(col) (curiosamente) pero no SELECT(col). Para satisfacer el pedido sobre empleado2 se crea una vista con exactamente las columnas autorizadas y se otorga SELECT ... WITH GRANT OPTION sobre la vista.

Bloque de administración (conexión como SYS).

```
-- sqlplus / as sysdba

-- Habilitar la aplicacion de limites de recursos definidos en perfiles.
-- Sin esta linea, CONNECT_TIME no surte efecto sobre director.
ALTER SYSTEM SET RESOURCE_LIMIT = TRUE;

-- Creacion de los tres usuarios
CREATE USER empleado1 IDENTIFIED BY "Empleado1_2026!"
  DEFAULT TABLESPACE users
  TEMPORARY TABLESPACE temp
  QUOTA 5M ON users;

CREATE USER empleado2 IDENTIFIED BY "Empleado2_2026!"
  DEFAULT TABLESPACE users
  TEMPORARY TABLESPACE temp
  QUOTA 5M ON users;

CREATE USER director IDENTIFIED BY "Director_2026!"
  DEFAULT TABLESPACE users
  TEMPORARY TABLESPACE temp
  QUOTA 5M ON users;

-- Rol que agrupa los privilegios comunes a empleado1 y empleado2.
CREATE ROLE empleados;

-- Privilegio de sistema para iniciar sesion.
GRANT CREATE SESSION TO empleado1, empleado2, director;

-- Asignacion del rol a los dos empleados.
GRANT empleados TO empleado1, empleado2;
```

```

-- Forzar a empleado1 a renovar la contraseña en el primer login.
ALTER USER empleado1 PASSWORD EXPIRE;

-- Perfil que limita la sesion de director a 20 minutos.
CREATE PROFILE director_profile LIMIT
  CONNECT_TIME 20;

ALTER USER director PROFILE director_profile;

```

Bloque de privilegios sobre objetos (conexión como PRACTICO1C).

```

-- sqlplus practico1c/practico1c

-- El rol empleados consulta automovil y categoria.
GRANT SELECT ON automovil TO empleados;
GRANT SELECT ON categoria TO empleados;

-- director: SELECT sobre todas las tablas del esquema, sin enumerarlas.
BEGIN
  FOR t IN (SELECT table_name FROM user_tables) LOOP
    EXECUTE IMMEDIATE 'GRANT SELECT ON ' || t.table_name || ' TO director';
  END LOOP;
END;
/

-- empleado1: DML completo sobre accidente.
GRANT INSERT, UPDATE, DELETE ON accidente TO empleado1;

-- empleado2: vista con las columnas autorizadas y SELECT con WITH GRANT OPTION.
CREATE OR REPLACE VIEW vw_cliente_datos AS
SELECT apellido, nombre, direccion
FROM cliente;

GRANT SELECT ON vw_cliente_datos TO empleado2 WITH GRANT OPTION;

```

Verificación.

```

-- Usuarios y perfil
SELECT username, account_status, profile
FROM dba_users
WHERE username IN ('EMPLEADO1', 'EMPLEADO2', 'DIRECTOR')
ORDER BY username;

-- Rol empleados
SELECT role FROM dba_roles WHERE role = 'EMPLEADOS';

-- Membresia del rol
SELECT grantee, granted_role
FROM dba_role_privs
WHERE grantee IN ('EMPLEADO1', 'EMPLEADO2', 'DIRECTOR')
ORDER BY grantee, granted_role;

-- Privilegios del rol empleados
SELECT grantee, owner, table_name, privilege
FROM dba_tab_privs
WHERE grantee = 'EMPLEADOS'
  AND owner = 'PRACTICO1C'
ORDER BY table_name, privilege;

-- Privilegios directos de empleado1 y director
SELECT grantee, owner, table_name, privilege

```

```

FROM dba_tab_privs
WHERE grantee IN ('EMPLEADO1', 'DIRECTOR')
  AND owner = 'PRACTICO1C'
ORDER BY grantee, table_name, privilege;

-- Vista de empleado2 y capacidad de delegar
SELECT grantee, owner, table_name, privilege, grantable
FROM dba_tab_privs
WHERE grantee = 'EMPLEADO2'
  AND owner = 'PRACTICO1C'
  AND table_name = 'VW_CLIENTE_DATOS';

-- Configuración del perfil de director
SELECT profile, resource_name, limit
FROM dba_profiles
WHERE profile = 'DIRECTOR_PROFILE'
  AND resource_name = 'CONNECT_TIME';

```

Por qué funciona

- En Oracle, la cuenta no puede conectarse sin `CREATE SESSION`; por eso se otorga ese privilegio de sistema explícitamente a los tres usuarios antes que cualquier otro `GRANT`.
- `ALTER USER empleado1 PASSWORD EXPIRE` deja la contraseña actual como expirada: en el próximo `CONNECT`, Oracle pide cambiarla. No es un cambio inmediato, es una marca que se aplica al siguiente intento de conexión.
- Los perfiles de Oracle (`CREATE PROFILE`) son el mecanismo nativo para imponer límites sobre recursos y contraseñas. `CONNECT_TIME` se mide en minutos y solo se hace efectivo si `RESOURCE_LIMIT = TRUE` a nivel sistema.
- Como Oracle no acepta `GRANT SELECT(col)`, la única forma idiomática de restringir a tres columnas es una vista. El `WITH GRANT OPTION` sobre la vista permite delegar exactamente esa proyección, no la tabla completa.
- La verificación se apoya en las vistas `DBA_*`: `DBA_USERS` (estado y perfil), `DBA_ROLE_PRIVS` (membresía), `DBA_TAB_PRIVS` (privilegios sobre tablas y vistas, con la columna `GRANTABLE` para confirmar `WITH GRANT OPTION`) y `DBA_PROFILES` (límites del perfil).

En MySQL 8+

Casi todo el ejercicio se traduce 1:1 a MySQL 8+, con dos diferencias importantes: (i) `empleado2` no necesita vista, porque MySQL sí admite `GRANT SELECT (col1, col2, ...) ... WITH GRANT OPTION`; (ii) el límite de 20 minutos de conexión continua *no tiene equivalente directo* en MySQL. La aproximación operativa más cercana es un evento programado que mate sesiones con tiempo de conexión mayor al límite, pero conceptualmente no es lo mismo que el `CONNECT_TIME` de Oracle. Se documenta abajo para dejar registro de la limitación.

```

USE practico1c_mysql;

-- Limpieza opcional
DROP USER IF EXISTS 'empleado1'@'localhost';
DROP USER IF EXISTS 'empleado2'@'localhost';
DROP USER IF EXISTS 'director'@'localhost';
DROP ROLE IF EXISTS 'empleados';

-- Rol comun
CREATE ROLE 'empleados';

-- Usuarios
CREATE USER 'empleado1'@'localhost' IDENTIFIED BY 'Empleado1_2026!';

```

```

CREATE USER 'empleado2'@'localhost' IDENTIFIED BY 'Empleado2_2026!';
CREATE USER 'director'@'localhost' IDENTIFIED BY 'Director_2026!';

-- Forzar a empleado1 a renovar password en el primer login
ALTER USER 'empleado1'@'localhost' PASSWORD EXPIRE;

-- Rol empleados con permisos sobre automovil y categoria
GRANT SELECT ON practico1c_mysql.automovil TO 'empleados';
GRANT SELECT ON practico1c_mysql.categoria TO 'empleados';

-- Asignacion del rol y activacion por defecto
GRANT 'empleados' TO 'empleado1'@'localhost', 'empleado2'@'localhost';
SET DEFAULT ROLE 'empleados'
TO 'empleado1'@'localhost', 'empleado2'@'localhost';

-- empleado1: DML completo sobre accidente
GRANT INSERT, UPDATE, DELETE
ON practico1c_mysql.accidente
TO 'empleado1'@'localhost';

-- empleado2: SELECT a nivel de columna sobre cliente, delegable.
-- En MySQL no hace falta crear una vista para esto.
GRANT SELECT (apellido, nombre, direccion)
ON practico1c_mysql.cliente
TO 'empleado2'@'localhost'
WITH GRANT OPTION;

-- director: SELECT sobre todas las tablas del esquema (SELECT *.* es exacto en MySQL).
GRANT SELECT ON practico1c_mysql.* TO 'director'@'localhost';

-- Limite de 20 minutos: no hay equivalente directo. Aproximacion via event
-- (requiere event_scheduler = ON). Mata sesiones de 'director' con tiempo de
-- conexion superior a 20 minutos.
-- TODO: confirmar con la catedra si la aproximacion via event es aceptable;
-- conceptualmente NO equivale a Oracle CONNECT_TIME, que se aplica antes de
-- la actividad del usuario.
CREATE EVENT IF NOT EXISTS kill_director_long_sessions
ON SCHEDULE EVERY 1 MINUTE
DO
  SELECT GROUP_CONCAT(CONCAT('KILL ', id) SEPARATOR '; ')
  FROM information_schema.processlist
  WHERE user = 'director' AND time > 20 * 60;

```

Qué cambia respecto de Oracle: (i) MySQL hace innecesaria la vista de empleado2: GRANT SELECT (col) ... WITH GRANT OPTION es soportado de forma nativa. (ii) CONNECT_TIME no tiene equivalente directo: wait_timeout solo controla *inactividad*, no la duración total. La aproximación operativa con un EVENT es funcional pero no idéntica conceptualmente, por eso queda marcada con % TODO.

6. Conclusión

El práctico ejercitó DCL en tres motores con modelos de seguridad bastante distintos. En MySQL, la identidad del usuario se forma con el par 'usuario'@'host' y los privilegios pueden afinarse a nivel de columna (incluido SELECT(col)); los límites de uso por hora se gestionan directamente con ALTER USER ... WITH. En PostgreSQL, "usuarios" y "roles" son la misma entidad y la herencia entre roles permite agrupar permisos comunes en un único contenedor, lo que simplifica la administración. En Oracle, hay que apoyarse en perfiles para limitar recursos como el tiempo de conexión, y ante la falta de GRANT SELECT(col) hay que recurrir a vistas para acotar la visibilidad de columnas.

Más allá de las diferencias entre motores, el ejercicio refuerza dos principios generales de seguridad: otorgar el privilegio mínimo necesario y delegar (con `WITH GRANT OPTION`) solo cuando hay una razón explícita para hacerlo. Las verificaciones con `SHOW GRANTS`, las vistas `information_schema` y `pg_*` en PostgreSQL y las vistas `DBA_*` en Oracle fueron clave para validar que cada `GRANT` y `REVOKE` produjo el efecto esperado.